

```
*Main> zip a b
[(1,"one"),(2,"two"),(3,"three"),(4,"four"),(5,"five")]

*Main> :t zip
zip :: [a] -> [b] -> [(a, b)]
```

The *let* expression defines the value of 'b' in the GHCi prompt. You can also define it in a way that's similar to the definition of the list 'a' in the source file.

The *lines* function takes input text and splits it at new lines:

```
*Main> let sentence = "First\nSecond\nThird\nFourth\nFifth"

*Main> lines sentence
["First","Second","Third","Fourth","Fifth"]

*Main> :t lines
lines :: String -> [String]
```

The *words* function takes input text and splits it on white space:

```
*Main> words "hello world"
["hello","world"]

*Main> :t words
words :: String -> [String]
```

The *map* function takes a function and a list, and applies the function to every element in the list:

```
*Main> map sumTwoInt a
[3,4,5,6,7]

*Main> :t map
map :: (a -> b) -> [a] -> [b]
```

The first argument to *map* is a function that is enclosed within parenthesis in the type signature $(a \rightarrow b)$. This function takes an input of type 'a' and returns an element of type 'b'. Thus, when operating over a list [a], it returns a list of type [b].

Recursion provides a means of looping in functional programming languages. The factorial of a number, for example, can be computed in Haskell, using the following code:

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n-1)
```

The definition of factorial with different input use cases is called pattern matching on the function. On running the above example with GHCi, you get the following output:

```
*Main> factorial 0
```

```
1
*Main> factorial 1
1
*Main> factorial 2
2
*Main> factorial 3
6
*Main> factorial 4
24
*Main> factorial 5
120
```

Functions operating on lists can also be called recursively. To compute the sum of a list of integers, you can write the *sumList* function as:

```
sumList :: [Int] -> Int
sumList [] = 0
sumList (x:xs) = x + sumList xs
```

The notation $(x:xs)$ represents a list, where 'x' is the first element in the list and 'xs' is the rest of the list. On running *sumList* with GHCi, you get the following:

```
*Main> sumList []
0
*Main> sumList [1,2,3]
6
```

Sometimes, you will need a temporary function for a computation, which you will not need to use elsewhere. You can then write an anonymous function. A function to increment an input value can be defined as:

```
*Main> (\x -> x + 1) 3
4
```

These are called Lambda functions, and the '\ ' represents the notation for the symbol Lambda. Another example is given below:

```
*Main> map (\x -> x * x) [1, 2, 3, 4, 5]
[1,4,9,16,25]
```

It is a good practice to write the type signature of the function first when composing programs, and then write the body of the function. Haskell is a functional programming language and understanding the use of functions is very important.  **END**

By: Shakthi Kannan

The author is a free software enthusiast and blogs at shakthimaan.com